Lecture 2 – AERM Macro

Florian Trouvain

University of Oxford

October 22, 2025

Neoclassical Model

The Neoclassical Model... Again!

- The neoclassical model is an extremely useful benchmark.
  - **Normative point of view:**
    - Think flexible vs. sticky prices in the business cycle literature.
    - Think misallocation vs. frictionless benchmark.
  - **Computational point of view:**
    - The neoclassical model is extremely well-behaved: unique, smooth, closed-form solution.
    - Most models nest the neoclassical model, which helps you (i) understand how your model works and (ii) find bugs in your code.
    - **Example:** Suppose I take your Aiyagari economy with income risk and make the income risk very small. It had better be that the interest rate is close to the discount rate, $r \approx \rho$.

Neoclassical Production

- Competitive goods and factor markets.
- Constant returns to scale.
- The final good is the numeraire and can be consumed or invested in capital, which depreciates at rate δ.

$$Y_t = AK_t^{\alpha} L_t^{1-\alpha}, \tag{1}$$

$$\dot{K}_t = Y_t - C_t - \delta K_t. \tag{2}$$

Neoclassical Household

- The representative agent maximizes utility (think of a dynasty if you like).
- Still a price taker!
- Holds assets B that pay a safe return r and earns labor income wL.
- Labor is supplied inelastically but grows at a constant rate g_L.

$$\max_{\{C_s\}} V_t = \int_t^\infty e^{-(\rho - g_L)s} \log(c_s)\, ds, \quad c_s := \frac{C_s}{L_s}, \tag{3}$$

$$\text{s.t. } \dot{B}_t = r_t B_t + w_t L_t - C_t. \tag{4}$$

Equilibrium

- **Production:**
  - Marginal products equal factor prices.
  - Firms rent capital from a competitive financial sector at rental rate R; financial sector intermediates between households and firms.

$$\Longrightarrow \quad \frac{\partial Y}{\partial K} = R, \qquad \frac{\partial Y}{\partial L} = w.$$

- **Household:**
  - Optimal consumption satisfies the Euler equation:

工厂支付资本租金率 R，而家庭获得的是资产回报率 delta。两者之间的差额是资本的折旧率

$$\boxed{\frac{\dot{c}}{c} = r - \rho.} \tag{5}$$

r 是资产的安全回报率（利率）
rho 是家庭的主观贴现率（耐心程度

如果利率 r（储蓄的奖励）大于家庭的耐心程度 rho，人们就会更愿意储蓄，导致消费增长

- **Financial sector:** $r = R - \delta, \qquad B = K.$

Steady State

- Given population growth, the steady state is defined only relative to a normalization $k := \frac{K}{L}$.

$$k_* = \left(\frac{\alpha A}{r_* + \delta} \right)^{\frac{1}{1-\alpha}}.$$

- Euler equation pins down r_*.
- c_*, y_*, and w_* follow immediately.

The Neoclassical Model... Again!

- The neoclassical model is an extremely useful benchmark
  - Normative point of view
    - Think flexible vs. sticky prices in business cycle
    - Think misallocation and cross-country income differences – frictionless benchmark
  - Computational point of view
    - Neoclassical model is extremely well behaved: unique, smooth, closed form solution
    - Most models nest the neoclassical model, which helps you i) understand how your model works and ii) find bugs in your code
  - Example: Suppose I take your Aiyagari economy with income risk, and I make the income risk very small. It better be that the interest rate is close to the discount rate $r \approx \rho$

Neoclassical Production

- Competitive goods and factor markets
- Constant returns to scale
- Final good is the numeraire and can be consumed or invested in capital, which depreciates at rate δ

$$Y = AK^{\alpha}L^{1-\alpha} \tag{6}$$

$$\dot{K} = Y - C - \delta K \tag{7}$$

Neoclassical Household

- Representative agent maximizes utility (think of dynasty if you like)
- Still, a price taker!
- Hold assets B which pay safe return r, and earn labor income wL
- Labor supplied inelastically but growing at constant rate g_L

$$\max_{\{C_s\}} V = \int_0^\infty e^{-(\rho - g_L)s} \log(c_s)\, ds, \quad c_s := \frac{C_s}{L_s} \tag{8}$$

$$\text{s.t. } \dot{B} = rB + wL - C \tag{9}$$

Equilibrium

- Production
  - Marginal product equals factor price
  - Rent capital from competitive financial sector that mediates between household and firms with rental rate R

$\implies \frac{\partial Y}{\partial K} = R, \frac{\partial Y}{\partial L} = w$

- Household
  - Optimal consumption obeys Euler equation

$$\frac{\dot{c}}{c} = r - \rho \tag{10}$$

- Financial sector: $r = R - \delta$, $B = K$

Steady State

- Given population growth, steady state only relative to some normalization, $k := \frac{K}{L}$

$$k_* = \left(\frac{\alpha A}{\rho + \delta} \right)^{\frac{1}{1-\alpha}}$$

- c_*, y_*, r_*, w_* follow immediately

Aiyagari Model

Consumption–Saving Choice with Risk

- An infinitely-lived household maximizes utility (think of a dynasty if you like).
- Holds assets b that pay a safe return r and earns labor income $w \cdot y$.
- The environment is stationary with constant aggregate variables (wage, interest rate, etc.).
- But it is not stationary for an individual household — y may fluctuate randomly:
  - $y \in \{0.5, 1.5\}$ and $w = 1$ for simplicity.
  - Markov transition matrix $\mathbb{P}(y' = y) = 0.8$ (some persistence).

Consumption–Saving Choice in Discrete Time

$$V = \max_{\{c_t\}} \mathbb{E}_t \sum_{t=0}^{\infty} \beta^t \frac{c_t^{1-\gamma}}{1-\gamma}, \tag{11}$$

$$\text{s.t.} \qquad b_{t+1} = (1+r)b_t + wy_t - c_t, \tag{12}$$

$$b_t \geq \underline{b} \quad \forall t. \tag{13}$$

Consumption–Saving Choice in Discrete Time: Some Terminology

- **Control variable:** c vs. **Endogenous state variable:** b vs. **Exogenous state variables:** everything else.
- Definition of a state variable: "A sufficient statistic for the dynamic evolution of the system."
- Methods carry over to a finite horizon, where the terminal condition is $b_T = 0$.
  - Note that as long as you do not know when you will die, the infinite-horizon setup is not so bad.
  - Opinion: I like the "dynastic assumption" — a micro-foundation for the infinite horizon (at least as good as warm-glow bequests or whatever else people do).

Consumption–Saving Choice in Discrete Time: Plethora of Implicit Assumptions

- There are many implicit assumptions:
  - Constant discounting.
  - Concave flow payoff.
  - Assumptions on continuity and boundedness to guarantee a maximum exists.
    - Uniqueness is much harder to prove.
  - Utility flow is additive (can be relaxed in useful ways, e.g., Epstein–Zin).
  - One consumption good — what if there are luxury goods?
  - Rational expectations:
    - The agent gets $\mathbb{E}_t$ "right."
    - The agent gets the evolution of aggregates "right."
    - Can be relaxed in various ways — behavioral economics, bounded rationality, ...

Consumption–Saving Choice in Discrete Time: Euler Equation

- In the absence of risk, we only need to satisfy the first-order conditions.
- Make sure you can derive the Euler equation:

$$c_t^{-\gamma} = \beta(1+r)\,\mathbb{E}_t[c_{t+1}^{-\gamma}].$$

Consumption–Saving Choice in Discrete Time: Euler Equation

- In the absence of risk, we only need to satisfy the first-order conditions.
- Make sure you can derive the Euler equation:

$$c_t^{-\gamma} = \beta(1+r)\,\mathbb{E}_t[c_{t+1}^{-\gamma}].$$

  - **Common mistake:** It should be "$\geq$" instead of "$=$" because of the borrowing constraint!
  - Without risk, this pins down consumption growth.
  - Together with the budget constraint, you can determine c_t for all t.
  - Linearization methods use this risk-free steady state and then perturb around it.
  - You can already see how this is a poor assumption from an individual point of view.

Global Solution & Bellman's Principle of Optimality

- "Policy" here means the optimal consumption path given the current states.

Quote: "An optimal policy has the property that, whatever the initial state and decision (i.e., control) are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision." (Bellman, 1957)

Bellman's Principle of Optimality

- **Observation 1:** We can split the sequence problem $\mathbb{E}_t \sum_t \beta^t u(c_t)$ into an optimal decision **today** versus the continuation value **tomorrow**.

$$V(b, y) = \max_c \left\{ \frac{c^{1-\gamma}}{1-\gamma} + \beta \, \mathbb{E} \, V(b', y') \right\} \tag{14}$$

- **Observation 2:** We are looking for functions $\{V(b, y), c(b, y)\}$.
- **Observation 3:** We have moved from time indices $\{t\}$ to states $\{b, y\}$.
  - Particularly convenient when there are infinitely many time periods but finitely many states.
  - Markovian structure implies we only need to track the current (b, y).
- **Observation 4:** Recursion depends on time running to infinity.

Value Function Iteration: We Are Looking for a Function

- While there is no obvious way to solve the sequence problem, we can solve the functional equation in (14) under standard regularity conditions
- Start with an initial guess, e.g. $V_0(b, y) = 0$.
- Use an iterative procedure where the subscript n tracks the iteration:

$$V_{n+1}(b, y) = \max_c \left\{ \frac{c^{1-\gamma}}{1-\gamma} + \beta \, \mathbb{E} \, V_n(b', y') \right\}.$$

- If $V_{n+1} \approx V_n$, you are done (given the regularity conditions).

Value Function Iteration: Using "Brute Force"

- The functional equation can be solved in several ways:
  - Root-finding methods based on Newton or quasi-Newton algorithms.
  - The endogenous grid method (Carroll, 2006), especially good for consumption–saving models.
- Here we use the "brute-force" value function iteration:
  - It always works!
  - Handles borrowing constraints, non-differentiabilities, kinks, etc.
  - Computationally costly:
    - Taking the max over a large state space can be slow.

Value Function Iteration: Grid Space & Stochastic Process

- Neither assets nor income are usually discrete variables.
- But an iterative procedure on a computer cannot run on an infinite set.
- Discretize the problem:
  - Define a grid space for assets — the "asset grid." : `b_grid = collect(LinRange(0, 5, 50))`.
  - Choosing bounds matters economically (we will revisit this).
- Observation: V_n converges arbitrarily close to the true V, but that V corresponds to the discretized (approximate) problem.
  - The true problem is continuous, so some approximation error is inevitable.
  - Trade-off: accuracy vs. speed.

Value Function Iteration: Build Payoff Function and Take the Max

- We want to find $V(b, y)$.
  1. Build the payoff function *u_c_vec* of size $N_b \times N_b \times N_y$.
  2. Apply the max operator:

$$v_{n+1}(b, y) = \max_{b'} \left\{ u(b' \mid b, y) + \beta \, \mathbb{E}[v_n(b', y') \mid y] \right\}.$$

  3. Important: the LHS uses step $n + 1$, the RHS uses n.
  4. Since v_n keeps changing, update the expectation: $\mathbb{E} v_{n+1} = \pi' v_{n+1}$.
  - **Observation:** It's easy to mix up rows and columns or reshape incorrectly.
  - **Warning:** The algorithm may still "converge" to something incorrect — always sanity-check the result.

Value Function Iteration: Grid Space

solve_v.jl

solve_v.jl > ...

```julia

# this file tests github autopilot
function solve_v(;params = params,v_int=v_int) # the semicolon is important here, it makes sure that the fu
                                                # you can also do solve_9(50,0,4,.8,...) but you have to remember the or
    # unpack parameters
    @unpack B_num,B_lower,B_upper,y_low,y_high,p_y,β,r,γ,max_iter_HJB,tol_HJB = params

    # set up grid space
    B_grid = collect(LinRange(B_lower,B_upper,B_num))
    y_grid = [y_low;y_high]
    pi_mat = [p_y 1-p_y;1-p_y p_y]
    y_num = size(y_grid)[1]

    convergence_v = false # boolean to check if converged

    # initialize value function -- could be any finite number, and doesn't have to be the same
    # if you want to converge quick, a good guess helps
    if v_int == 0
        v_0 = -ones(y_num*B_num) # states_y X states_B
    else
        v_0 = v_int
    end
    v_vec = copy(v_0)
    E_v = reshape(reshape(v_vec,B_num,y_num)*pi_mat',B_num*y_num)
    policy_function = zeros(B_num*y_num) # this one will be an index, where the index refers to the optimal

    # make payoff vector -- will be of size B_num*B_num*y_num (note ``curse of dimensionality'')
    # quick function to deal with negative consumption, important note: γ is defined above, and then used i
    function u(c)
        if c>0
            return (c^(1-γ))/(1-γ)
        else
            return -1000000  # we just want something very negative here, so that in the end no one is ever
        end
    end
```

Value Function Iteration: Payoff Function

```
# make payoff vector -- will be of size B_num*B_num*y_num (note ``curse of dimensionality'')
# quick function to deal with negative consumption, important note: γ is defined above, and then used insi
function u(c)
    if c>0
        return (c^(1-γ))/(1-γ)
    else
        return -1000000  # we just want something very negative here, so that in the end no one is ever go
    end
end

y_grid_large = kron(y_grid,ones(B_num*B_num))
c_vec = kron(ones(y_num),(1+r)*kron(B_grid,ones(B_num))-kron(ones(B_num),B_grid)) + y_grid_large # when pr
u_c_vec = u.(c_vec)

## start the iterative procedure -- maximum iter is max_iter_HJB
for l = 1:max_iter_HJB
    # blow up the value function from a B_num*y_num vector to a B_num*B_num*y_num vector, so you can take
    v_continue=[]
    for i=1:y_num
        v_continue = [v_continue; kron(ones(B_num),E_v[B_num*(i-1)+1:B_num*(i)])]
    end

    # combine the payoff vector with the continuation value (payoff always the same but v is being updated
    v_vec_big = u_c_vec + β*v_continue
    v_vec_old = copy(v_vec) # copy matters here instead of simply setting v_vec_old = (v_vec), figure out

    for i=1:B_num*y_num # we have to iterate over all states (B*y)
        v_vec[i] = findmax(v_vec_big[1+B_num*(i-1):B_num*i])[1] # findmax returns a tuple (max_value, inde
    end
```

- It need not be stored as a vector — you could use multidimensional tuples.
- What if you don't know how `ones()` and `kron(a, b)` work?

Value Function Iteration: Computing the Expectation

- We have a conditional expectation:

$$\mathbb{E}[V(b, y') \mid y] = \sum_{y'} \pi(y' \mid y)\, V(b, y').$$

- Does it matter that we write b instead of b' here?
- In our case:

$$\Pi_{y'|y} = \begin{pmatrix} \rho & 1-\rho \\ 1-\rho & \rho \end{pmatrix}.$$

Value Function Iteration: Take the Max and Compute Expectation

```
for l = 1:max_iter_HJB
    # blow up the value function from a B_num*y_num vector to a B_num*B_num*y_num vector, so you
    v_continue=[]
    for i=1:y_num
        v_continue = [v_continue; kron(ones(B_num),E_v[B_num*(i-1)+1:B_num*(i)])]
    end

    # combine the payoff vector with the continuation value (payoff always the same but v is bein
    v_vec_big = u_c_vec + β*v_continue
    v_vec_old = copy(v_vec) # copy matters here instead of simply setting v_vec_old = (v_vec), fi

    for i=1:B_num*y_num # we have to iterate over all states (B*y)
        v_vec[i] = findmax(v_vec_big[1+B_num*(i-1):B_num*i])[1] # findmax returns a tuple (max_va
    end

    # get the expected value -- pi_mat is the stochastic matrix that matters now
    E_v = reshape(reshape(v_vec,B_num,y_num)*pi_mat',B_num*y_num) # you might be able to write th
    if findmax(abs.(v_vec-v_vec_old))[1]< tol_HJB # check for convergence
        # if converged, get the policy function and break the loop
        for i=1:B_num*y_num
            policy_function[i] = Int(findmax(v_vec_big[1+B_num*(i-1):B_num*i])[2])
        end
        convergence_v = true # boolean to check if converged

        break
    end
end
# turn policy function from index into actual consumption
```

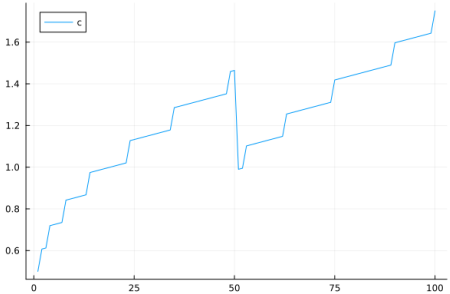
Value Function Iteration: Get the Policy Function

- Who cares about V?
- What we really want is $c(b, y)$ — the optimal consumption policy given states.
- Final step using your solved V:
  - Iterate one final time, but instead of taking the max and storing the value,
  - track the index of whichever element in the vector was maximizing.
  - Example: at the borrowing constraint, you consume all your income:
    - Say $b_{\text{current}} = 0$, $y = 0.5$, and optimally $b' = 0$ (as solved).

    $\implies$ $c(b = 0, y = 0.5) = 0.5$.
  - Once you have $c(b, y)$, you have solved the household's problem!

Value Function Iteration: Some examples

- `b_grid = LinRange(0,5,50)`
- Plot $c(b, y)$ — how did I order b and y?
- Kind of okay, but not great — monotone, but we would like to see something "smoother".



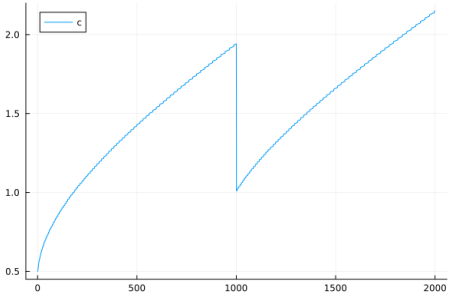
Value Function Iteration: Some examples

- `b_grid = LinRange(0,5,500)`
- Costly — takes a lot longer to solve (0.5 vs. 5 sec). Toni Whited tells me you have to solve a model more than 10k times, so...
- Still not perfect — note the effect of the upper boundary.

these are a point on the vectors
not asset position

Value Function Iteration: Some examples

- `b_grid = LinRange(0,15,1500)` — runs for 21 seconds.
- Setting up the grid has important economic implications:
  - Here: an ad-hoc zero borrowing constraint.
  - The upper bound matters too $\implies$ it may force households to consume even though they would save more if the grid were larger. How do I know? Theory!



Value Function Iteration: Final comment

- More advanced methods out there
  - Endogenous grid point method (Carroll, 2006).
  - Clever spacing of the grid (policy function is roughly linear for large wealth).
  - Continous time finite difference scheme (Achdou et al., 2022)
  - Many tricks to make value function iteration faster
- But the basic approach here is useful because it is:
  - Pedagogical.
  - Robust — it generally works while other methods are context-specific.

Aggregates & Data

- We did a lot, but we are still missing:
  - Aggregation
  - Data
- Essential to combine micro-dynamics with macroeconomic outcomes.

Solve for Stationary Distribution: Stochastic Matrices

- Stochastic Matrix:
  - Elements are non-negative.
  - Rows sum to one.
  - Every row represents a conditional probability distribution.

$$\Pi_y = \begin{pmatrix} 0.8 & 0.2 \\ 0.2 & 0.8 \end{pmatrix}$$

- Regularity conditions $\implies$ a unique stationary distribution $x \in \mathbb{R}^2_+$ (what is it?).
- Solve $x = \Pi_y' x$ or iterate $x_{n+1} = \Pi_y' x_n$.

Solve for Stationary Distribution

- Suppose we have a unit measure of households (infinitely many so LLN applies).
- Note that we have solved for $c(b, y)$, and we made assumptions about $y' \mid y$.
- Classic approach: draw y's and simulate thousands of households.
- Better: construct the transition matrix Π which depends on:
  - Evolution of **income**: $\begin{pmatrix} 0.8 & 0.2 \\ 0.2 & 0.8 \end{pmatrix}$, and
  - Evolution of **assets** (endogenous), e.g. rows like $[0\ 0\ \ldots\ 1\ 0]$.
- Iterate until convergence: $x_\infty = \mu :=$ stationary distribution:

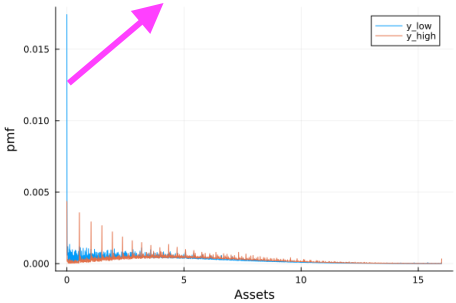
$$x_{n+1} = \Pi' x_n \tag{15}$$

Code Snippet: Stationary Income Distribution

```
x = copy(x_int)
for k = 1:max_iter_KFE
    x_old = copy(x)
    x = trans_mat' * x
    if findmax(abs.(x-x_old))[1] < tol_KFE
        println("CONV stationary distn. after $k iterations.")
        break
    end
end
```

Stationary Distribution: A Bit Clunky

- Looks reasonable to you?



- Take my word for it — continuous-time methods much nicer.

Equilibrium Interest Rate

- Find the interest rate such that household demand for safe assets equals firm supply of capital.
- GE equilibrium condition:

$$B = K. \tag{16}$$

- Aggregate asset supply: $B = \sum_{b,y} b \cdot \mu(b, y)$.
- Neoclassical capital demand: $K = \frac{\alpha Y}{r+\delta}$.

Equilibrium Interest Rate: Aiyagari (1994)

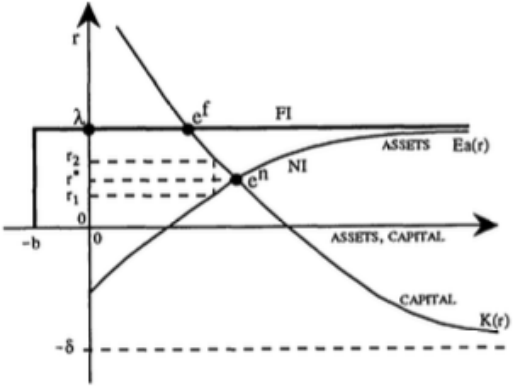


FIGURE IIb
Steady-State Determination

Figure 1: Taken from Aiyagari (1994)

Find r_{ss} using Bisection

- Finally, we iterate over guesses for the equilibrium interest rate r:
  - Household problem + income process $\implies$ stationary distribution μ.
  - Stationary distribution implies supply of assets $\int b_i \, di$ (or sum over discrete states).
  - Neoclassical production implies demand for capital $K(r, \mathbb{E}y)$.

Bisection: Code Snippet

```julia
for k = 1:100
    sol_v = solve_v(params=params_next,v_int=sol_v.v_vec) # comes from the code in "solve_v.jl"
    (mat,x) = solve_g(;sol_v=sol_v,x_int=x,params=params_next)
    sol_K = comp_K_S_D(;dist_ss=x,params = params_next)

    if sol_K.K_D > sol_K.K_S
        r_low = (r_up+r_low)/2
        r_up = r_up
        params_next = params_bl(B_num=B_num_final_sim,B_upper=B_upper_final_sim,r=(r_low+r_up)/2)
    else
        r_low = r_low
        r_up = (r_up+r_low)/2
        params_next = params_bl(B_num=B_num_final_sim,B_upper=B_upper_final_sim,r=(r_low+r_up)/2)
    end
    println("r_low = ",r_low,", r_up = ",r_up)
    println("sol_K.K_D-sol_K.K_S = ",sol_K.K_D-sol_K.K_S)
    if abs.(sol_K.K_D-sol_K.K_S)<params_next.tol_r
        r_ss = (r_low+r_up)/2
        K_ss = (sol_K.K_D+sol_K.K_S)/2
        println("Conv K,r.")
        break
    end
end
```

Find r_{ss} using Bisection

- Example result: $r_{ss} = 0.0915$.
- Contrast with the complete-market benchmark $r_{\text{CM}} = \frac{1}{\beta} - 1 = 0.111$.
- Why is $r_{ss} < r_{\text{CM}}$?
- Why might I have chosen a low β, which implies a rather high interest rate?

Preview Next Lecture

1. Richer income process
   - One way in which data matters a lot.
2. Simulate the model to understand regression results vs. structural relationships:
   - Interpret regression coefficients through the lens of the model.
   - Compare partial- and general-equilibrium effects of policy changes.

References

Achdou, Y., J. Han, J.-M. Lasry, P.-L. Lions, and B. Moll (2022). Income and wealth distribution in macroeconomics: A continuous-time approach. *The review of economic studies 89*(1), 45–86.